Yavuzlar Bootcamp Konuları Araştırma Ödevi

Feyzanur Yüksel

Front-End

Öncelikle Front-End slaytından başladım.CSS ve HTML ayrımını pek anlamadığımdan araştırdım. HTML web sayfasının iskeletini oluştururken CSS daha iyi bir görünüm sağlamasına yarıyor. HTML sayfanın yapıdan, düzenden CSS ise stilden, renkten sorumlu oluyor.

JavaScript bu konunun neresinde diye düşündüğüm için ufak bir araştırma yaptım. HTML, CSS ile birlikte çalışan bu dil web sayfasındaki aksiyonlardan, etkileşimlerden sorumlu.

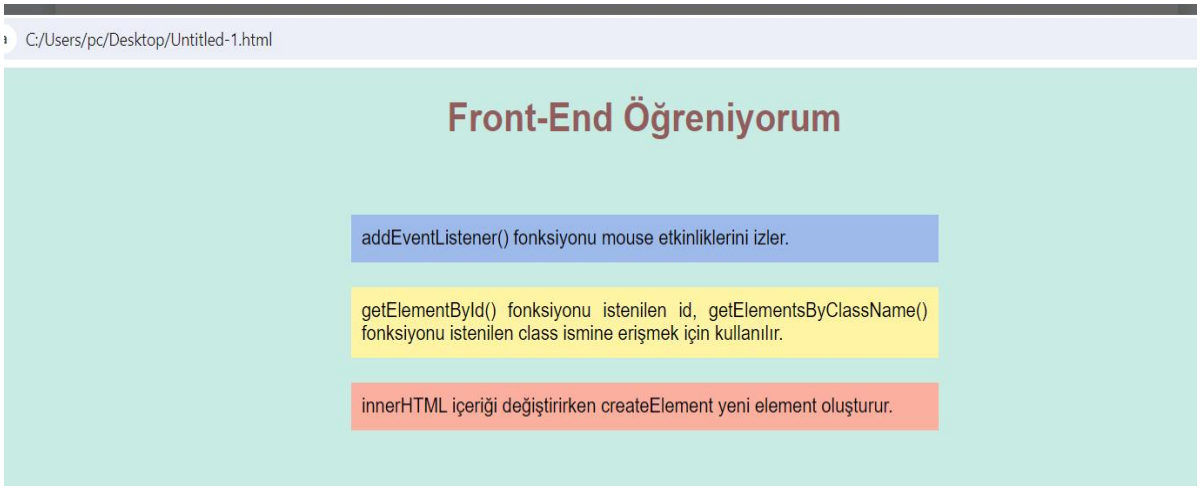
JavaScript konusunda çok az bilgim olduğu için biraz daha üstünde durdum.

Edindiğim bilgiler;

-JavaScript bir programlama diliyken HTML işaretleme, CSS stil dili. Bu nedenle JavaScript temelinde aşina olduğumuz diziler, döngüler, fonksiyonlar mevcut.

-DOM konusuna baktığımda bunun aslında C++ programlama dilindeki veri yapılarıyla oldukça benzer olduğunu düşündüm. Binary tree ve linked list ile çok benzerlik gösteriyor. Biraz araştırdığımda DOM içinde de düğümler(node) olduğunu, istediğimiz veriyi fonksiyon ile çağırabildiğimizi, değiştirebildiğimizi, ekleme yapabildiğimizi gördüm.

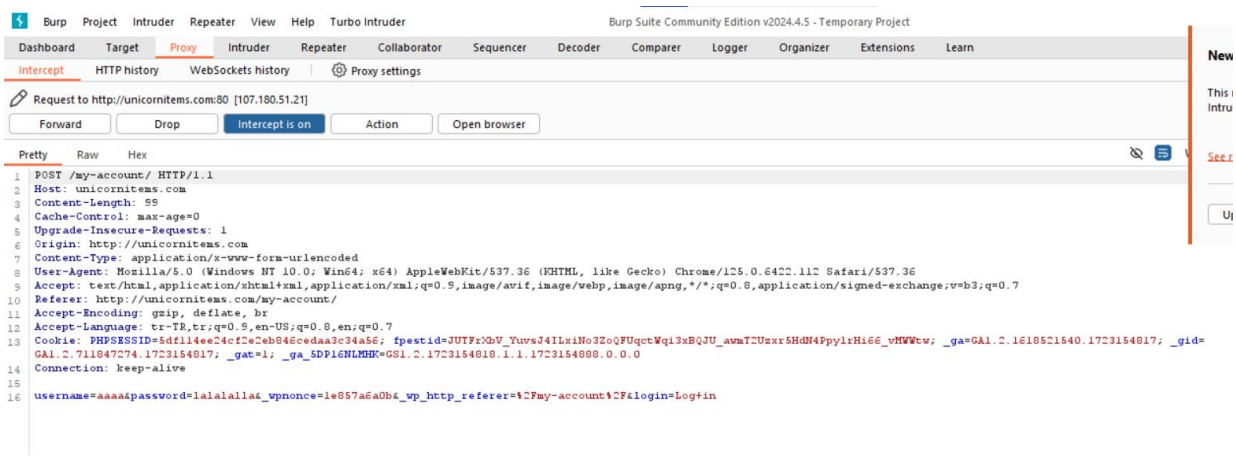
HTML, CSS ve JavaScript konusunda kendimi bir üst seviyeye çıkarabilmek için basit, bilgilendirici bir web sayfası hazırlamaya karar verdim.



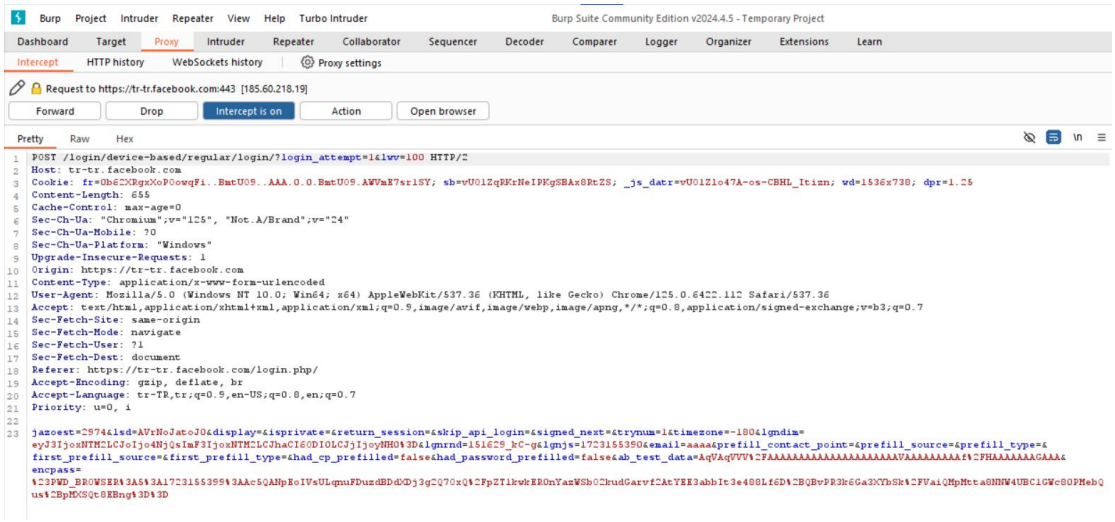
```
<!DOCTYPE html>
<html>

<head>
  <style>
    body {
      background: #c8ebe4;
      text-align: center;
      font-family: 'Open Sans', sans-serif;
    }

    .container {
      display: flex;
      flex-direction: column;
      align-items: center;
      margin: 50px;
    }

    .box {
      list-style-type: none;
      padding: 0;
      margin: 10px 0;
      text-align: justify;
      width: 540px;
    }

    .box1 {
      background-color: #9dbaea;
    }

    .box2 {
      background-color: #FFF4A3;
    }

    .box3 {
      background-color: #FAAF9F;
    }

    .box-item {
      padding: 10px;
    }
  </style>
</head>

<body>
  <center>
    <h1 style="color:rgb(148, 93, 93);">Front-End Öğreniyorum</h1>
    <div class="container">
      <ul class="box box1">
        <li class="box-item">addEventListener() fonksiyonu mouse etkinliklerini izler.</li>
      </ul>
      <ul class="box box2">
        <li class="box-item">getElementById() fonksiyonu istenilen id, getElementsByClassName() fonksiyonu istenilen class ismine erişmek için kullanılır.
        </li>
      </ul>
      <ul class="box box3">
        <li class="box-item">innerHTML içeriği değiştirirken createElement yeni element oluşturur.</li>
      </ul>
    </div>
  </center>
</body>

</html>
```

Web Nedir

Sıradaki slayta çalışmaya başladım. HTTP ve HTTPS arasındaki en büyük farkın şifreleme olduğunu gördüm. Bunu kendim görmek istedim. HTTP kullanan bir sitede hesap girişi yapmayı denedim. İsteği Burp Suite üzerinden incelediğimde şifre, username gibi bilgilerin rahatça okunabildiğini görüyorum. Aynı işlemi HTTPS kullanan bir sitede yaptığımda ise username ve şifrenin anlaşılmadığını, şifrelenerek iletildiğini görüyorum.



HTTP kullanan site



HTTPS kullanan site

Bu iki istekte de slaytta bahsedilen istek başlıklarını, sunucuya veri gönderdiğim için en başta yazan POST isteğini, HTTP 2 versiyonunun HTTP 1.1 versiyonundan daha gelişmiş olduğunu görebiliriz.

QUIC

Slayta devam ettiğimde, QUIC adlı bir konuyla karşılaştım. Araştırdığımda QUIC'in (Quick UDP Internet Connections) başlangıçta Google tarafından geliştirilen ve şimdi bir internet standardı haline gelen yeni bir taşıma katmanı protokolü olduğunu öğrendim. QUIC TCP'yi değiştirmeyi amaçlıyor ve HTTP/3'ün temelini oluşturuyor.

HTTPS over TCP+TLS: HTTPS, TCP kullanarak client ve server arasında çalışıyor. Bağlantı güvenliğini TLS sağlıyor. Ancak TCP, el sıkışma işlemleri nedeniyle daha yüksek RTT (Round-Trip Time) ile bağlantı kuruyor.

HTTPS over QUIC: HTTPS, TCP yerine UDP tabanlı QUIC protokolüyle çalışıyor. QUIC, TLS 1.3'ü entegre ederek client-server bağlantısını daha düşük RTT ile daha hızlı kuruyor.

SSH

SSH ağ protokolüne kendim giriş yapmayı denedim. Port 2220 ile bağlantı kurup doğru şifreyi yazdığımda bağlanabildim.

Client-Side Rendering vs Server-Side Rendering

Slaytta Client-Side Rendering vs Server-Side Rendering şemaları vardı ancak tam anlamıyla bu kavramları anlayamadım ve araştırmaya karar verdim.

Anladığım kadarıyla ikisi de web sayfalarını işliyor. Client-Side Rendering bu işlemi tarayıcıda, istemci tarafında oluştururken Server-Side Rendering web sayfasını sunucuda oluşturuyor. Client-Side Rendering sayfayı etkileşimli olarak yavaş yüklerken Server-Side Rendering aynı sayfayı sadece gösterir ancak daha hızlı yükleme imkanı sağlar.

Web Sunucuları

Slayttan çalışmaya devam ettiğimde web sunucularını gördüm. Apache sunucusunun daha genişletilebilir, özelleştirilebilir olduğunu, bunun yanında yüksek kaynak tükettiğini anladım. Nginx sunucusu ise yüksek performans ve düşük kaynak tüketimine sahip.

Apache ve Nginx sunucularının bilindik hangi uygulamalar/siteler tarafından kullanıldığını merak ettiğim için ufak bir araştırma yaptım.
Apache sunucusu için:
Wikipedia,Adobe,LinkedIn,PayPal,BBC
Nginx sunucusu için:
Netflix,Airbnb,GitHub,Pinterest,Instagram

Bunu araştırırken fark ettim ki bu sunucular aynı zamanda beraber de kullanılabilir. Nginx hızlı ve verimli içerik sunumu için kullanılırken, Apache dinamik içerik sunumu için tercih edilir.Bu şekilde siteler yüksek performans, güvenilirlik ve esneklik sağlamış olur.

Veritabanı Sunucuları

Slayta devam ettiğimde veritabanı sunucu türleri hakkında olan kısmı anlamadım ve araştırmaya karar verdim.

-İlişkisel Veritabanları(Relational/SQL Database)
Veriler, satırlar ve sütunlar içeren tablolar şeklinde depolanır.
Tablo Tabanlı Yapı,İlişkiler,SQL,Veri Bütünlüğü,Optimizasyon

-Zaman Serisi Veritabanları(Time Series Database)
Zaman damgalı verileri depolamak, yönetmek ve analiz etmek için optimize edilmiş veritabanlarıdır.
Yüksek Yazma Performansı,Veri Sıkıştırma,Zaman Serisi İndeksleme

-NoSQL Veritabanları
Yapılandırılmış veya yarı yapılandırılmış verileri depolamak ve yönetmek için kullanılan, genellikle esnek bir veri modeli sunan veritabanı türleridir.
Esneklik,Yatay Ölçeklenebilirlik,Belge Tabanlı,Anahtar-Değer Veritabanları

Web Uygulamasının Çalışma Mimarisi

Bir web uygulamasının çalışma mimarisini araştırdığımda üç ana kısımdan oluştuğunu gördüm. Bunların ilki olan Front-End kullanıcıların tarayıcı üzerinde görebildiği, etkileşimde bulunabildiği kısımdır. Back-End ise kullanıcıdan gelen istekleri alır, veritabanıyla iletişim kurar ve sonuçları geri gönderir. Veritabanı ise Verilerin saklandığı yerdir. Back-End, veriyi almak veya kaydetmek için buraya sorgular gönderir.

Bir web uygulamasının nasıl işlediğini adım adım anlatmak istersem önce client(istemci) tarayıcı üzerinden bir HTTP isteği gönderir. Bu istek Back-End kısmında sunucuya gönderilir. Sunucu isteğin gereğini yapar isterse veritabanından gerekli verileri alır veya veriyi saklar. Backend, isteği yanıtlayarak veriyi HTTP yanıtı olarak Front-End kısmına geri gönderir. Front-End ise bu cevabı kullanıcıya yani client tarafına gösterir.

Modern Web Çalışma Mimarisi

Modern web çalışma mimarisine baktığımda çok karışık bir şema gördüm. Bunu açıklığa kavuşturmak için araştırma yaptım. Şemanın sırasına ve anladığıma göre:

Kullanıcının tarayıcıdan bir URL girmesiyle işlemler başlar. Tarayıcı bu URL'yi IP adresini öğrenmek için DNS sorgusu yapar.

1) DNS: URL'yi IP adresine çevirir ve tarayıcıya geri gönderir.

2) Load Balancer: Trafiği yönlendirir ve yükü dengelemek için isteği uygun web uygulama sunucusuna iletir.

3) Web Uygulama Sunucuları: Dinamik içerik oluşturur ve gerekli verileri toplar.

4) Veritabanı: Web uygulama sunucusu, gerekli verileri almak için veritabanına sorgu gönderir.

5) Caching Service(Önbellekleme Servisi): Eğer bu dinamik içerik daha önce oluşturulmuş ve önbelleğe atıldıysa web sunucusu önbelleğe bakar ve içeriği doğrudan buradan alır. Eğer içerik önbellekte yoksa, web sunucusu veritabanı, işlem sonucunu önbelleğe yazar. Bu, gelecekteki isteklerde aynı işlemi tekrar yapmayarak önbellekten alınmasını sağlar.

6-a) Job Queue (İş Kuyruğu): Uzun süren veya arka planda çalışan görevler varsa, bu görevler iş kuyruğuna eklenir ve iş sunucuları tarafından işlenir.

6-b) Job Servers (İş Sunucuları): İş kuyruğundaki görevleri işler ve sonuçları gerekli yerlere iletir.

7) Full-Text Search Service: Kullanıcı bir arama yapıyorsa arama sorgusu full-text search servisine iletilir. Bu servis, arama sonuçlarını hızla bulur ve web uygulama sunucusuna geri gönderir.

8) Services (Servisler): Web uygulamasındaki belirli görevleri yerine getiren bağımsız bileşenlerdir.

9-a) Data Firehouse: Eğer uygulama veri toplama işlemleri yapıyorsa, bu veri firehouse üzerinden toplanabilir ve işlenebilir.

9-b) Copy of Data (Veri Kopyası): Verinin güvenli bir şekilde saklanması ve olası veri kaybı durumlarında geri yüklenmesi için kullanılır.

9-c) Data Warehouse: büyük veri kümelerinin toplandığı, organize edildiği ve analiz için hazırlandığı merkezi bir depolama alanıdır.

10) Cloud Storage (Bulut Hafıza): Veri kopyaları, bulut depolama hizmetlerinde saklanabilir. Bu, veriye her yerden erişim ve yüksek erişilebilirlik sağlar.

11) CDN (Content Delivery Network): Statik içerikleri önbellek üzerinde tutar. Eğer istenen içerik CDN üzerinde önbelleğe alınmışsa doğrudan CDN'den geri döner. Önbelleğe alınmamışsa istek web uygulama sunucusuna iletilir.